

Week 2 Notes — Regression with Multiple Input Variables

Andrew Ng — Machine Learning Specialization (Course 1: Supervised Machine Learning)

Contents

Multiple Features (Multiple Linear Regression)	4
Introduction	4
Definition	4
Why It Matters	4
Core Concept	4
Formula	4
Explain Symbols	4
Working (Step-by-Step)	5
Example	5
Visualization	5
Common Mistakes	5
Interview Points	5
Quick Revision	5
Connection to Next Concept	5
Vectorization	6
Introduction	6
Definition	6
Why It Matters	6
Core Concept	6
Formula	6
Explain Symbols	6
Working (Step-by-Step)	6
Example	7
Visualization	7
Common Mistakes	7
Interview Points	7
Quick Revision	7
Connection to Next Concept	7
Gradient Descent for Multiple Linear Regression	8
Introduction	8
Definition	8
Why It Matters	8
Core Concept	8

Formula	8
Explain Symbols	8
Working (Step-by-Step)	9
Example	9
Visualization	9
Common Mistakes	9
Interview Points	9
Quick Revision	9
Connection to Next Concept	10
Feature Scaling	11
Introduction	11
Definition	11
Why It Matters	11
Core Concept	11
Formula	11
Explain Symbols	11
Working (Step-by-Step)	12
Example	12
Visualization	12
Common Mistakes	12
Interview Points	12
Quick Revision	13
Connection to Next Concept	13
Checking Gradient Descent for Convergence	14
Introduction	14
Definition	14
Why It Matters	14
Core Concept	14
Formula	14
Explain Symbols	14
Working (Step-by-Step)	14
Example	15
Visualization	15
Common Mistakes	15
Interview Points	15
Quick Revision	15
Connection to Next Concept	15
Choosing the Learning Rate	16
Introduction	16
Definition	16
Why It Matters	16
Core Concept	16
Formula	16
Explain Symbols	16
Working (Step-by-Step)	16

Example	17
Visualization	17
Common Mistakes	17
Interview Points	17
Quick Revision	17
Connection to Next Concept	17
Feature Engineering	18
Introduction	18
Definition	18
Why It Matters	18
Core Concept	18
Formula	18
Explain Symbols	18
Working (Step-by-Step)	18
Example	19
Visualization	19
Common Mistakes	19
Interview Points	19
Quick Revision	19
Connection to Next Concept	19
Polynomial Regression	20
Introduction	20
Definition	20
Why It Matters	20
Core Concept	20
Formula	20
Explain Symbols	20
Working (Step-by-Step)	20
Example	21
Visualization	21
Common Mistakes	21
Interview Points	21
Quick Revision	21
Connection to Next Concept	21
Week 2 Master Summary	22

Multiple Features (Multiple Linear Regression)

Introduction

So far, a model that predicts a price from just one input (like house size) is limited — real-world outcomes usually depend on **many factors at once**. Multiple linear regression extends simple linear regression so the model can learn from several input features together, not just one. This is the natural next step once a single feature isn't enough to explain the output well.

Definition

Multiple linear regression is a supervised learning model that predicts a target value y as a **weighted sum** of multiple input features $x_1, x_2, \dots, x_n$, plus a bias term.

Why It Matters

Real data is rarely explained by a single variable. A house's price depends on its size, number of bedrooms, number of floors, and age — not just one of these. Multiple features let the model capture more of the real pattern in the data, which usually improves prediction accuracy.

Core Concept

Instead of one weight w and one feature x , we now have **one weight per feature**. Each feature contributes to the prediction according to how important it is (its weight), and all contributions are added together along with a base value b .

Formula

$$f_{\vec{w},b}(\vec{x}) = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

Compact (dot product) form:

$$f_{\vec{w},b}(\vec{x}) = \vec{w} \cdot \vec{x} + b$$

Explain Symbols

Symbol	Meaning
$x_1, x_2, \dots, x_n$	Individual input features (e.g., size, bedrooms, floors, age)
$\vec{x}$	Vector containing all features of one training example
$w_1, w_2, \dots, w_n$	Weight (importance) assigned to each feature
$\vec{w}$	Vector containing all weights
b	Bias term (base value when all features are zero)
n	Number of features
$f_{\vec{w},b}(\vec{x})$	Model's predicted output

Working (Step-by-Step)

1. Collect the value of every feature for a given example $\rightarrow$ form vector $\vec{x}$.
2. Multiply each feature by its own weight ($w_1x_1, w_2x_2, \dots$).
3. Add all the weighted terms together.
4. Add the bias term b .
5. The result is the predicted value $f_{\vec{w},b}(\vec{x})$.

Example

Predicting house price using 4 features: - x_1 = size (sq ft) = 1200 - x_2 = bedrooms = 3 - x_3 = floors = 1 - x_4 = age (years) = 10

If $\vec{w} = [50, 20, 5, -2]$ and $b = 30$ (units in \$1000s):

$$f = 50(1200) + 20(3) + 5(1) - 2(10) + 30$$

(Note: in practice, size would use a much smaller weight after scaling — this is just to illustrate the formula.)

Visualization

Refer to the course's "Multiple Features" slide — it shows a table of training examples across several feature columns, feeding into the same linear formula.

Common Mistakes

- Confusing n (number of features) with m (number of training examples).
- Forgetting that $\vec{w}$ and $\vec{x}$ must have the same length (n).
- Assuming a bigger weight always means a more important feature — this is only true if features are on comparable scales.

Interview Points

- **Q: What's the difference between simple and multiple linear regression?** A: Simple uses one feature; multiple uses two or more, combined as a weighted sum.
- **Q: What does the bias term represent?** A: The predicted value when all features are zero — a baseline offset.

Quick Revision

- Multiple features = one weight per feature.
- Formula: $f_{\vec{w},b}(\vec{x}) = \vec{w} \cdot \vec{x} + b$
- n = number of features, $\vec{w}$ and $\vec{x}$ are vectors of length n .
- Prediction = sum of (weight $\times$ feature) + bias.

Connection to Next Concept

Computing $\vec{w} \cdot \vec{x} + b$ using a loop over n features is slow. The next concept, **vectorization**, shows how to compute this same formula much faster.

Vectorization

Introduction

Once a model has many features, calculating predictions and gradients one feature at a time becomes slow — especially with large datasets. Vectorization solves this by using **linear algebra operations** instead of loops, letting the computer do many calculations at once.

Definition

Vectorization is the technique of expressing a computation as operations on entire vectors/arrays (using optimized numerical libraries like NumPy) instead of writing explicit **for** loops over individual elements.

Why It Matters

- Loops in Python are slow for large n .
- Vectorized code uses hardware-level parallelism (SIMD, multi-core, GPU), making computations dramatically faster.
- Code also becomes shorter and easier to read.

Core Concept

Instead of writing a loop that multiplies each w_j by x_j and adds them one at a time, vectorization computes the entire dot product $\vec{w} \cdot \vec{x}$ in a single optimized operation.

Formula

Without vectorization (loop):

$$f = \sum_{j=1}^n w_j x_j + b$$

With vectorization (NumPy):

```
f = np.dot(w, x) + b
```

Explain Symbols

Symbol	Meaning
w, x	NumPy arrays representing $\vec{w}$ and $\vec{x}$
$\text{np.dot}(w, x)$	Computes $\vec{w} \cdot \vec{x}$ internally, optimized in low-level code
j	Index running from 1 to n over features

Working (Step-by-Step)

1. Store all weights in one NumPy array w and all features in one array x .
2. Call `np.dot(w, x)` — NumPy computes the sum of element-wise products internally, using optimized C/parallel code.

3. Add the bias b to get the final prediction.
4. The same idea extends to gradient descent: gradients for all n weights are updated in one vectorized step instead of n separate loop iterations.

Example

```
import numpy as np
w = np.array([0.5, 20, 5, -2])
x = np.array([1200, 3, 1, 10])
b = 30
f = np.dot(w, x) + b
```

This single line replaces a 4-iteration loop — and would replace an even bigger loop if n were in the thousands.

Visualization

Refer to the course’s “Vectorization Part 1 & 2” slides, which compare the loop-based computation timeline versus the parallel, single-step vectorized computation timeline.

Common Mistakes

- Writing manual loops out of habit even when w and x are NumPy arrays — this wastes the speed benefit.
- Mismatched array shapes/lengths between w and x causing silent broadcasting errors.
- Forgetting that vectorization changes **speed**, not the **mathematical result** — the answer is identical either way.

Interview Points

- **Q: Why is vectorized code faster than a Python loop?** A: It uses low-level, parallelized operations (SIMD/multi-core) instead of Python’s slower interpreted loop.
- **Q: Does vectorization change the model’s output?** A: No — it only changes how fast the same computation is performed.

Quick Revision

- Vectorization = replace loops with array/matrix operations.
- Key tool: `np.dot()` for dot products.
- Same math, much faster execution — critical for large n and large datasets.
- Also speeds up gradient descent updates across all weights simultaneously.

Connection to Next Concept

Vectorization makes gradient descent computations efficient. The next section applies gradient descent to the multiple linear regression formula introduced earlier.

Gradient Descent for Multiple Linear Regression

Introduction

Now that the model can use multiple features (and compute them efficiently via vectorization), it still needs a way to **learn** the best values of $\vec{w}$ and b from data. Gradient descent, previously used for one feature, extends naturally to many features.

Definition

Gradient descent for multiple linear regression is an iterative optimization algorithm that simultaneously updates each weight w_j (for $j = 1$ to n) and the bias b in the direction that reduces the cost function $J(\vec{w}, b)$.

Why It Matters

With multiple features, there isn't one single weight to tune — there are n weights plus a bias. Gradient descent gives a systematic way to adjust **all of them together** so the model's predictions get closer to the actual training values.

Core Concept

For every weight w_j , compute how much the cost changes if w_j changes slightly (its partial derivative/gradient), then move w_j a small step in the direction that decreases cost. Do the same for b . Repeat until the cost stops decreasing meaningfully.

Formula

Repeat until convergence (all updates happen **simultaneously**):

$$w_j := w_j - \alpha \frac{\partial J(\vec{w}, b)}{\partial w_j} \quad \text{for } j = 1, \dots, n$$
$$b := b - \alpha \frac{\partial J(\vec{w}, b)}{\partial b}$$

Where the partial derivatives are:

$$\frac{\partial J(\vec{w}, b)}{\partial w_j} = \frac{1}{m} \sum_{i=1}^m (f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)}) x_j^{(i)}$$
$$\frac{\partial J(\vec{w}, b)}{\partial b} = \frac{1}{m} \sum_{i=1}^m (f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)})$$

Explain Symbols

Symbol	Meaning
α	Learning rate — how big each update step is
m	Number of training examples

Symbol	Meaning
$x_j^{(i)}$	Value of feature j in training example i
$y^{(i)}$	Actual (true) target value for example i
$f_{\vec{w},b}(\vec{x}^{(i)})$	Model's prediction for example i
$J(\vec{w}, b)$	Cost function (measures how wrong the model is overall)

Working (Step-by-Step)

1. Initialize all weights $w_1, \dots, w_n$ and b (often to 0).
2. Compute predictions for all training examples using current $\vec{w}, b$.
3. Compute the gradient (partial derivative) for each w_j and for b .
4. Update every w_j and b **at the same time** using the formulas above.
5. Repeat steps 2–4 until the cost J stops decreasing significantly (convergence).

Example

With 2 features and a small dataset, each iteration: - Computes the error (prediction — actual) for every example. - Multiplies that error by each feature's value, averages it → gradient for that feature's weight. - Nudges w_1, w_2 , and b slightly in the direction that reduces total error. - Repeats thousands of times until predictions stabilize.

Visualization

Refer to the course's contour plot / 3D bowl-shaped cost function surface for multiple features — gradient descent takes steps “downhill” toward the minimum, just like in the single-feature case, but now in a higher-dimensional space.

Common Mistakes

- Updating w_1 using the **already-updated** w_2 instead of updating all parameters simultaneously.
- Using a learning rate α that's too large → cost diverges instead of decreasing.
- Forgetting to feature-scale before running gradient descent (covered next) — this can make convergence very slow.

Interview Points

- **Q: Why must updates to all weights happen simultaneously?** A: Because each gradient is computed using the *old* parameter values; updating one early would corrupt the gradient calculations for the others in the same step.
- **Q: How is gradient descent for multiple features different from one feature?** A: Mathematically it's the same idea repeated for each w_j — only now there are n weight-update equations running in parallel instead of one.

Quick Revision

- Same gradient descent idea, extended to n weights + 1 bias.

- All parameters updated **simultaneously** each iteration.
- Gradient = average of (error $\times$ corresponding feature value).
- Vectorization (previous topic) makes this fast even for large n .

Connection to Next Concept

Gradient descent converges much faster and more reliably when features are on similar scales. This motivates the next topic: **feature scaling**.

Feature Scaling

Introduction

When features have very different ranges (e.g., house size in the thousands vs. number of bedrooms as a small single digit), gradient descent can become slow and inefficient. Feature scaling fixes this by putting all features on a similar numerical range before training.

Definition

Feature scaling is a preprocessing technique that transforms features so they have comparable ranges (or comparable variance), making gradient descent converge faster and more reliably.

Why It Matters

If one feature ranges from 0–2000 and another from 0–5, the cost function becomes a long, narrow “elongated bowl” shape. Gradient descent then zig-zags slowly across the narrow direction instead of heading straight to the minimum. Scaling makes the bowl more circular/symmetric, so gradient descent converges in a much more direct path.

Core Concept

Bring every feature roughly into the same range (commonly around -1 to 1) using one of a few standard techniques, so no single feature dominates the gradient calculations just because of its scale.

Formula

1. Simple (min-max-like) scaling by max value:

$$x_j^{scaled} = \frac{x_j}{\max(x_j)}$$

2. Mean normalization:

$$x_j^{scaled} = \frac{x_j - \mu_j}{\max(x_j) - \min(x_j)}$$

3. Z-score normalization (standardization):

$$x_j^{scaled} = \frac{x_j - \mu_j}{\sigma_j}$$

Explain Symbols

Symbol	Meaning
x_j	Original (unscaled) value of feature j
μ_j	Mean (average) of feature j across all training examples
σ_j	Standard deviation of feature j

Symbol	Meaning
$\max(x_j), \min(x_j)$	Maximum and minimum values of feature j in the dataset

Working (Step-by-Step)

1. For each feature column, compute its mean μ_j and standard deviation σ_j (or min/max, depending on method).
2. Apply the chosen scaling formula to every value of that feature.
3. Repeat for every feature independently.
4. Train the model (e.g., run gradient descent) on the scaled features instead of raw values.
5. Remember: predictions on new/unseen data must be scaled using the **same** μ_j, σ_j computed from training data.

Example

Feature “size” ranges from 300 to 2000 sq ft, mean $\mu = 1150$, std dev $\sigma = 400$. For a house of size 1500:

$$x^{scaled} = \frac{1500 - 1150}{400} = 0.875$$

Now “size” is on a similar numeric scale to something like “bedrooms” (which might range from -1 to 1 after scaling too).

Visualization

Refer to the course’s contour plots comparing an elongated, narrow cost function (unscaled features) versus a more circular, symmetric one (scaled features) — showing why gradient descent zig-zags less after scaling.

Common Mistakes

- Scaling the entire dataset (train + test) together, causing **data leakage** — always fit scaling parameters (μ, σ) on training data only.
- Forgetting to apply the same scaling to new/unseen inputs at prediction time.
- Assuming feature scaling changes the final answer’s meaning — it doesn’t change what the model learns conceptually, just how efficiently it gets there.

Interview Points

- **Q: Why does feature scaling speed up gradient descent?** A: It makes the cost function’s contours more circular/symmetric, so each gradient step points more directly toward the minimum instead of zig-zagging.
- **Q: What’s the difference between mean normalization and z-score normalization?** A: Mean normalization uses the range (max–min) as the denominator; z-score uses the standard deviation, which is generally more robust to outliers.

Quick Revision

- Feature scaling puts all features on comparable ranges.
- Common methods: divide by max, mean normalization, z-score normalization.
- Speeds up and stabilizes gradient descent convergence.
- Must use the *same* scaling parameters for training and future predictions.

Connection to Next Concept

Once features are scaled and gradient descent is running, we need a way to check whether it's actually working correctly — that's covered next in **checking gradient descent for convergence**.

Checking Gradient Descent for Convergence

Introduction

Running gradient descent doesn't guarantee it's working well — it might be too slow, stuck, or even diverging. This topic covers a simple, practical way to visually confirm that gradient descent is successfully minimizing the cost function.

Definition

Convergence checking is the process of monitoring the cost function $J(\vec{w}, b)$ over training iterations to confirm it is consistently decreasing and has leveled off, indicating gradient descent has found (or nearly found) the minimum.

Why It Matters

Without checking convergence, you can't tell whether: - the model needs more iterations to finish learning, - the learning rate is poorly chosen, or - gradient descent has already found a good solution and further training is wasted computation.

Core Concept

Plot the cost J against the number of iterations (a **learning curve**). If gradient descent is working correctly, this curve should **decrease after every iteration** and eventually flatten out.

Formula

No new formula — this reuses the cost function computed during training:

$$J(\vec{w}, b) = \frac{1}{2m} \sum_{i=1}^m (f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)})^2$$

An alternative to visual inspection is an **automatic convergence test**: declare convergence if J decreases by less than a small threshold ϵ (e.g., 0.001) in one iteration.

Explain Symbols

Symbol	Meaning
$J(\vec{w}, b)$	Cost after a given iteration, using current parameters
Iteration number	x-axis of the learning curve plot
ϵ	Small threshold used to automatically decide “good enough” convergence

Working (Step-by-Step)

1. During training, record $J(\vec{w}, b)$ at the end of every iteration (or every few iterations).
2. Plot J (y-axis) against iteration number (x-axis).
3. Inspect the curve:

- Steadily decreasing $\rightarrow$ gradient descent is working.
 - Flattening near the end $\rightarrow$ close to convergence.
 - Increasing or oscillating $\rightarrow$ something is wrong (often the learning rate).
4. Optionally, use an automatic threshold check instead of manually reading the plot.

Example

If after 100 iterations J drops from 500 to 50 and keeps shrinking slowly, and by iteration 400 it barely moves (e.g., $12.01 \rightarrow 12.00$), that's a sign gradient descent has essentially converged — more iterations won't help much.

Visualization

Refer to the course's "Checking gradient descent for convergence" plot — a curve that starts high and steadily flattens as iterations increase; this single graph is the main tool for this topic.

Common Mistakes

- Stopping training too early, mistaking a temporary flattening for full convergence.
- Not noticing an **increasing** cost curve, which usually signals the learning rate is too high.
- Relying only on a fixed number of iterations without ever checking the curve.

Interview Points

- **Q: How do you know gradient descent has converged?** A: The cost function's learning curve flattens out and further iterations produce negligible decrease.
- **Q: What does it mean if the cost curve increases during training?** A: The learning rate is very likely too large, causing gradient descent to overshoot the minimum.

Quick Revision

- Plot J vs iteration number — this is the "learning curve."
- Should always decrease; flattening = convergence.
- Increasing/oscillating cost = learning rate problem.
- Automatic convergence test: stop when J 's decrease is smaller than a threshold ϵ .

Connection to Next Concept

If the learning curve shows problems (increasing cost, extremely slow decrease), the most common fix is adjusting the learning rate — covered next.

Choosing the Learning Rate

Introduction

The learning rate α controls how big each gradient descent step is. Picking it well is one of the most practical skills in training models — too small wastes time, too large can break training entirely.

Definition

The **learning rate** (α) is a hyperparameter that scales the size of each parameter update step during gradient descent.

Why It Matters

- Too small $\alpha \rightarrow$ gradient descent takes tiny steps $\rightarrow$ training is extremely slow.
- Too large $\alpha \rightarrow$ gradient descent overshoots the minimum $\rightarrow$ cost may oscillate or even diverge (increase without bound).
- Choosing α well is often the difference between a model that trains successfully and one that doesn't converge at all.

Core Concept

Test a range of learning rate values, observe the cost-vs-iteration curve for each, and pick the largest value that still reliably decreases the cost every iteration.

Formula

Recall the general update rule:

$$w_j := w_j - \alpha \frac{\partial J(\vec{w}, b)}{\partial w_j}$$

A common practical strategy is to try values in roughly a **3x increasing pattern**:

$$\alpha = 0.001, 0.003, 0.01, 0.03, 0.1, 0.3, 1, \dots$$

Explain Symbols

Symbol	Meaning
α	Learning rate — step size multiplier
$\frac{\partial J}{\partial w_j}$	Gradient direction/magnitude for parameter w_j

Working (Step-by-Step)

1. Start with a small learning rate (e.g., 0.001) and run a handful of iterations.
2. Plot the cost-vs-iteration curve; confirm cost decreases every iteration.
3. Gradually increase α (roughly 3x each time) and re-check the curve.
4. If cost starts increasing/oscillating at some value, that α is too large.
5. Pick a value just below the point where problems start — this gives fast but stable convergence.

6. As a quick debugging trick: try a **very tiny** α ; if cost still doesn't decrease every iteration, there's likely a bug in the code (not the learning rate).

Example

Trying $\alpha = 0.3$ shows the cost curve oscillating up and down — too large. Trying $\alpha = 0.1$ shows smooth, fast decrease — this becomes the chosen learning rate.

Visualization

Refer to the course's side-by-side learning curve comparisons for different α values — one smoothly decreasing curve (good α) versus one oscillating/increasing curve (too-large α).

Common Mistakes

- Assuming a bigger learning rate is always faster — past a certain point, it breaks convergence entirely.
- Confusing a bug in the gradient/cost code with a “bad learning rate” — always test with a tiny α first to isolate the cause.
- Using the same fixed α for every dataset without re-checking the learning curve.

Interview Points

- **Q: What happens if the learning rate is too large?** A: Gradient descent can overshoot the minimum repeatedly, causing the cost to oscillate or diverge.
- **Q: What's a good debugging strategy if cost isn't decreasing at all?** A: Try an extremely small learning rate — if cost still doesn't decrease, the issue is a bug, not α .

Quick Revision

- α controls step size in gradient descent.
- Too small = slow; too large = diverges/oscillates.
- Try values increasing $\sim 3\times$ at a time; watch the learning curve.
- Pick the largest stable value for fastest reliable convergence.

Connection to Next Concept

With training now efficient and stable, the next step is improving what the model is actually trained on — starting with **feature engineering**, which creates smarter inputs from existing data.

Feature Engineering

Introduction

Sometimes the raw features given in a dataset don't capture the pattern well on their own. Feature engineering means using domain knowledge or intuition to create new, more meaningful features from existing ones — often making a simple model perform much better.

Definition

Feature engineering is the process of creating new input features (by transforming or combining existing ones) using intuition or domain knowledge, in order to help a model learn patterns more effectively.

Why It Matters

A model can only find relationships that its features actually expose. If the *true* relationship depends on a combination of raw features (like $\text{area} = \text{length} \times \text{width}$), giving the model that combination directly can make learning far easier and more accurate than hoping it discovers the pattern on its own.

Core Concept

Look at the raw features and ask: “Is there a more meaningful quantity I can compute from these that better relates to the target?” Then add that computed quantity as a brand-new feature.

Formula

General idea — given existing features x_1 and x_2 , engineer a new feature x_3 :

$$x_3 = x_1 \times x_2$$

(This is just one example — engineered features can also be ratios, differences, or other domain-relevant combinations.)

Explain Symbols

Symbol	Meaning
x_1, x_2	Original raw features (e.g., frontage length, depth of a lot)
x_3	New engineered feature (e.g., total area = $x_1 \times x_2$)

Working (Step-by-Step)

1. Understand what the raw features physically represent.
2. Identify a combination that might be more directly related to the target variable.
3. Compute this new feature for every training example.
4. Add it to the feature set (can keep or drop the original features).
5. Retrain the model including this new feature and compare performance.

Example

Predicting house price with features: x_1 = frontage (lot width), x_2 = depth (lot length). Engineered feature:

$$x_3 = x_1 \times x_2 = \text{lot area}$$

“Area” is likely a much stronger predictor of price than frontage and depth considered separately — the model can now directly use area as a feature.

Visualization

Refer to the course’s example comparing a model using only x_1, x_2 separately vs. one that also includes the engineered area feature x_3 — the latter fits the price pattern better.

Common Mistakes

- Removing the original raw features when they still carry useful information on their own.
- Creating overly complex engineered features without justification, increasing risk of overfitting.
- Forgetting that engineered features usually still need **feature scaling** if their range differs a lot from other features.

Interview Points

- **Q: What is feature engineering and why is it useful?** A: It’s creating new, more informative features from raw data using domain knowledge, which can make patterns easier for the model to learn.
- **Q: Give an example of feature engineering.** A: Multiplying lot frontage and depth to create an “area” feature for house price prediction.

Quick Revision

- Feature engineering = creating smarter features from existing ones.
- Uses domain knowledge/intuition, not just raw data as-is.
- Example: area = frontage $\times$ depth.
- Often dramatically improves model performance without changing the algorithm.

Connection to Next Concept

One particularly powerful and common application of feature engineering is creating **polynomial features** — this leads directly into **polynomial regression**.

Polynomial Regression

Introduction

Not every real-world relationship is a straight line. Polynomial regression extends linear regression to fit **curves** by engineering new features that are powers of an existing feature (like x^2 , x^3), while still using the same linear regression machinery underneath.

Definition

Polynomial regression is a form of regression where the model includes polynomial terms (powers) of the original feature(s) as additional inputs, allowing it to fit non-linear/curved relationships using the linear regression framework.

Why It Matters

Many real relationships are curved, not straight (e.g., price vs. size might flatten out or curve upward). A plain straight-line model can't capture this — polynomial regression lets a “linear” model represent curves by feeding it powers of x as extra features.

Core Concept

Take an existing feature x and create new features x^2 , x^3 , etc. Then apply ordinary linear regression on this expanded feature set — the model is still linear *in the weights*, even though the resulting curve is non-linear in x .

Formula

$$f_{\vec{w},b}(x) = w_1x + w_2x^2 + w_3x^3 + \dots + b$$

Or with a square-root feature (another common alternative when data flattens out):

$$f_{\vec{w},b}(x) = w_1x + w_2\sqrt{x} + b$$

Explain Symbols

Symbol	Meaning
x	Original raw feature
$x^2, x^3, \dots$	Engineered polynomial features (powers of x)
$w_1, w_2, w_3, \dots$	Weights for each polynomial term
b	Bias term

Working (Step-by-Step)

1. Start with the original feature x .
2. Create new features: x^2 , x^3 (or other transforms like $\sqrt{x}$), depending on the shape of the data.
3. Treat these as additional input features alongside (or instead of) x .

4. **Feature scaling becomes critical here** — x^3 can be enormously larger in range than x , which would badly slow gradient descent if left unscaled.
5. Run ordinary linear regression (with gradient descent) on this expanded feature set.
6. Compare curve fits to choose the right degree (too high a degree risks overfitting).

Example

If house price vs. size looks like it curves and flattens at higher sizes, using:

$$f(x) = w_1x + w_2x^2 + b$$

lets the model bend to match this pattern — something a plain straight line ($f(x) = w_1x + b$) could never do.

Visualization

Refer to the course's plots comparing a straight-line fit vs. a quadratic/cubic curve fit vs. a square-root curve fit on the same non-linear dataset — showing how higher-degree terms let the curve bend to match the data.

Common Mistakes

- Forgetting feature scaling — x^3 can have a vastly different range than x , badly slowing convergence.
- Choosing too high a polynomial degree, which can overfit and produce a wiggly, unrealistic curve.
- Believing polynomial regression is a “different algorithm” — it's still linear regression, just with cleverly engineered features.

Interview Points

- **Q: Is polynomial regression still linear regression?** A: Yes — it's linear in the weights; the non-linearity comes entirely from the engineered polynomial features, not the algorithm.
- **Q: What's a key preprocessing step before running polynomial regression with gradient descent?** A: Feature scaling, since powers of x can have drastically different ranges.

Quick Revision

- Polynomial regression = linear regression + power-based engineered features ($x^2, x^3, \sqrt{x}, \dots$).
- Lets a linear model fit curved relationships.
- Feature scaling is essential due to widely different feature ranges.
- Degree choice matters — too high risks overfitting.

Connection to Next Concept

With the ability to fit both straight lines and curves, the next stage in the course moves toward **classification problems** (e.g., logistic regression) — where the target is a category rather than a continuous number.

Week 2 Master Summary

Topic	One-Line Takeaway
Multiple Features	Prediction = weighted sum of several features + bias
Vectorization	Use array operations (<code>np.dot</code>) instead of loops for speed
Gradient Descent (Multi-feature)	Same update rule, applied simultaneously to all weights + bias
Feature Scaling	Bring features to similar ranges to speed up convergence
Checking Convergence	Plot cost vs. iterations; should steadily decrease and flatten
Choosing Learning Rate	Test increasing α values; pick largest stable one
Feature Engineering	Create smarter features using domain knowledge (e.g., area)
Polynomial Regression	Add power terms (x^2, x^3) to fit curves with linear regression